MATERI PRAKTIKUM PENGAYAAN: PEMBAHASAN MENDALAM STORED PROCEDURE, UDF, CURSOR & ERROR HANDLING

Modul pengayaan ini disusun secara komprehensif untuk melengkapi Materi Praktikum 4 (USER DEFINE FUNCTION & STORE PROCEDURE) pada mata kuliah Sistem Manajemen Basis Data Lanjut (SMBD Lanjut). Fokus modul ini adalah memberikan pemahaman mendalam mengenai arsitektur logika bisnis terkompilasi di sisi server (RDBMS), perbedaan hakiki antara User Defined Function (UDF) dan Stored Procedure, klasifikasi parameter IN, OUT, dan INOUT, mekanisme iterasi baris per baris dengan CURSOR, penanganan eksepsi (Error Handling) menggunakan HANDLER dan SIGNAL SQLSTATE, serta penerapan skenario otomasi transaksi kasir minimarket 3NF lengkap dengan 10 latihan soal bertingkat dan lampiran kunci jawaban khusus asisten/dosen.

1. KONSEP DASAR & ARSITEKTUR STORED ROUTINES

Dalam rekayasa sistem basis data relasional enterprise, Stored Routine adalah blok kode prosedural SQL yang disimpan secara permanen di dalam katalog basis data (data dictionary) dan dieksekusi langsung oleh database engine di sisi server. Stored Routine terdiri dari dua jenis utama: User Defined Function (UDF) dan Stored Procedure.

1.1 Mengapa Memindahkan Logika Bisnis ke Database?

- **Reduksi Lalu Lintas Jaringan (Network Traffic Optimization):** Pada arsitektur tradisional, aplikasi klien harus mengirimkan belasan kueri SELECT, INSERT, dan UPDATE secara bolak-balik melalui jaringan. Dengan Stored Routine, aplikasi klien cukup mengirimkan satu perintah CALL, dan seluruh rangkaian kueri dieksekusi langsung di memori server, memangkas latensi jaringan secara dramatis.
- **Sentralisasi Logika Bisnis (Centralized Business Logic):** Logika transaksi kritis—seperti pemotongan stok barang, kalkulasi diskon member, dan akumulasi poin reward—disentralisasi di dalam basis data. Semua aplikasi klien (aplikasi POS kasir, aplikasi mobile e-commerce, web portal admin) memanggil subrutin yang sama, menghilangkan risiko inkonsistensi perhitungan.
- **Kompilasi dan Eksekusi Cepat (Execution Plan Caching):** Stored Routine dikompilasi satu kali dan rencana eksekusinya disimpan di dalam cache memori RDBMS. Eksekusi berulang tidak memerlukan proses parsing dan optimasi sintaks dari awal, menghasilkan performa eksekusi kueri yang jauh lebih cepat.
- **Lapisan Keamanan Berlapis (Security & Privilege Isolation):** Aplikasi kasir tidak perlu diberi izin akses langsung (INSERT/UPDATE) ke tabel fisik PRODUK dan CUSTOMER. Cukup berikan izin EXECUTE pada Stored Procedure checkout, sehingga basis data terlindungi dari manipulasi langsung.

1.2 Perbedaan Hakiki Function vs Stored Procedure

Parameter Komparasi	User Defined Function (UDF)	Stored Procedure
Nilai Kembalian (Return)	Wajib mengembalikan satu nilai skalar tunggal melalui klausa RETURNS dan pernyataan RETURN.	Tidak mengembalikan nilai melalui klausa RETURN; dapat mengembalikan nilai via parameter OUT/INOUT atau result set.
Cara Pemanggilan	Dipanggil di dalam ekspresi kueri SQL biasa (contoh: SELECT fc_name(), WHERE fc_name() > 0).	Dipanggil secara mandiri menggunakan perintah eksplisit: CALL proc_name(param).
Manipulasi Data Fisik (DML)	Sangat dibatasi; secara standar tidak boleh melakukan modifikasi data (INSERT/UPDATE/DELETE) yang mengubah state tabel.	Mendukung eksekusi DML penuh, transaksi (COMMIT/ROLLBACK), dynamic SQL, dan pemanggilan procedure lain.
Tujuan Utama	Melakukan kalkulasi matematis, manipulasi teks, atau transformasi data yang	Mengeksekusi alur kerja bisnis lengkap, batch processing, dan transaksi multi-tabel.

menghasilkan nilai tunggal.

2. USER DEFINED FUNCTION (UDF) DI MYSQL

User Defined Function memungkinkan perancang basis data membuat fungsi matematis, pemformatan string, dan logika kustom yang belum disediakan oleh fungsi bawaan (built-in functions) MySQL.

2.1 Sintaks Lengkap Deklarasi Function

```
CREATE [DEFINER = user]
FUNCTION [IF NOT EXISTS] func_name ([param_name param_type, ...])
RETURNS return_type
[DETERMINISTIC | NOT DETERMINISTIC]
[CONTAINS SQL | NO SQL | READS SQL DATA | MODIFIES SQL DATA]
[SQL SECURITY { DEFINER | INVOKER }]
BEGIN
   DECLARE variabel_lokal tipe_data [DEFAULT nilai];
   -- Logika komputasi prosedural
   RETURN nilai_kembalian;
END;
```

2.2 Karakteristik Penentu Function

- **DETERMINISTIC:** Fungsi selalu menghasilkan nilai keluaran yang sama jika diberikan argumen input yang sama. Contoh: fungsi penambah angka atau pemotong string. Wajib dideklarasikan pada sistem dengan binary logging aktif.
- **NOT DETERMINISTIC:** Fungsi dapat menghasilkan nilai berbeda meskipun argumennya sama (contoh: fungsi yang membaca waktu saat ini NOW() atau membaca data tabel fisik yang dinamis).
- **READS SQL DATA:** Memberitahukan kepada query optimizer bahwa fungsi hanya membaca data dari tabel (SELECT) tanpa melakukan mutasi data.

2.3 Contoh Implementasi UDF: Generator Kode & Poin Belanja

```
-- 1. Mengubah Delimiter agar titik-koma di dalam blok BEGIN-END tidak memutus eksekusi
DELIMITER $$

-- 2. Function Generator Nomor Nota Transaksi Otomatis (Format: YYMMDDXXXX)
CREATE FUNCTION fcGenerateNoNota(tgl DATE)
RETURNS CHAR(10)
DETERMINISTIC
READS SQL DATA
BEGIN
   DECLARE tgl_prefix CHAR(6);
   DECLARE last_seq INT DEFAULT 0;
   DECLARE new_nota CHAR(10);

   -- Mengambil format YYMMDD (contoh: 2022-08-03 -> '220803')
   SET tgl_prefix = DATE_FORMAT(tgl, '%y%m%d');

   -- Mencari nomor urut transaksi terakhir pada tanggal tersebut
   SELECT IFNULL(MAX(CAST(RIGHT(nonota, 4) AS UNSIGNED)), 0) INTO last_seq
   FROM nota
```

```
    WHERE LEFT(nonota, 6) = tgl_prefix;

    -- Menggabungkan tanggal prefix dengan 4 digit nomor urut baru
    SET new_nota = CONCAT(tgl_prefix, LPAD(last_seq + 1, 4, '0'));
    RETURN new_nota;
END$$

-- 3. Function Kalkulasi Poin Belanja (1 Poin tiap kelipatan Rp 10.000)
CREATE FUNCTION fcHitungPoinBelanja(total_belanja INT)
RETURNS INT
DETERMINISTIC
NO SQL
BEGIN
    DECLARE jml_poin INT DEFAULT 0;
    IF total_belanja >= 10000 THEN
        SET jml_poin = FLOOR(total_belanja / 10000);
    END IF;
    RETURN jml_poin;
END$$

DELIMITER ;

-- Pengujian Eksekusi:
SELECT fcGenerateNoNota('2022-08-03') AS nota_baru;
SELECT fcHitungPoinBelanja(14800) AS poin_diperoleh;
```

3. STORED PROCEDURE: MANAJEMEN PARAMETER & TRANSAKSI

Stored Procedure dirancang untuk menangani tugas transaksi yang kompleks. Parameter pada Stored Procedure dapat dikonfigurasikan ke dalam tiga mode interaksi data:

3.1 Tiga Mode Parameter Stored Procedure

Mode Parameter	Arah Aliran Data	Karakteristik & Perilaku Teknis
IN (Default)	Pemanggil -> Procedure	Nilai dilewatkan ke dalam procedure sebagai input. Procedure dapat membaca nilai tersebut, namun perubahan variabel di dalam procedure tidak akan mempengaruhi nilai pemanggil.
OUT	Procedure -> Pemanggil	Variabel diinisialisasi sebagai NULL di dalam procedure. Procedure mengisinya dengan hasil perhitungan, dan nilai tersebut dikembalikan ke pemanggil saat procedure selesai.
INOUT	Pemanggil <-> Procedure	Variabel membawa nilai awal dari pemanggil ke dalam procedure, nilainya dimodifikasi oleh procedure, dan nilai hasil akhirnya dikembalikan ke pemanggil.

3.2 Contoh Implementasi Parameter IN, OUT, dan INOUT

```
DELIMITER $$

-- Procedure dengan parameter IN dan OUT: Registrasi Member Baru & Mengembalikan Kode Barunya
CREATE PROCEDURE spRegistrasiMember(
    IN p_nama VARCHAR(50),
    IN p_tgllahir DATE,
    IN p_jenkel CHAR(1),
    IN p_alamat VARCHAR(100),
    IN p_hp VARCHAR(15),
    OUT p_kode_dihasilkan CHAR(5)
)
BEGIN
    DECLARE last_num INT DEFAULT 0;

    -- Generate Kode Member Baru (Format M0001, M0002, dst)
    SELECT IFNULL(MAX(CAST(RIGHT(kodecus, 4) AS UNSIGNED)), 0) INTO last_num FROM customer;
    SET p_kode_dihasilkan = CONCAT('M', LPAD(last_num + 1, 4, '0'));

    -- Simpan ke tabel fisik customer
    INSERT INTO customer (kodecus, namacus, tgllahir, jenkelcus, almtcus, hpcus, pointterkumpul)
    VALUES (p_kode_dihasilkan, p_nama, p_tgllahir, p_jenkel, p_alamat, p_hp, 0);
END$$

-- Procedure dengan parameter INOUT: Menghitung Tambahan Poin Bonus Transaksi
CREATE PROCEDURE spHitungBonusLoyalitas(
    IN p_status_member VARCHAR(20),
    INOUT p_poin INT
)
BEGIN
    IF p_status_member = 'VIP' THEN
        SET p_poin = p_poin + 20; -- Bonus tambahan 20 poin
    ELSEIF p_status_member = 'REGULER' THEN
        SET p_poin = p_poin + 5;  -- Bonus tambahan 5 poin
    END IF;
END$$

DELIMITER ;

-- Pengujian Eksekusi Parameter OUT:
CALL spRegistrasiMember('Rina Melati', '1998-07-21', 'P', 'Yogyakarta', '081299887766', @kode_baru);
SELECT @kode_baru AS 'ID Member Terdaftar';

-- Pengujian Eksekusi Parameter INOUT:
SET @poin_awal = 10;
CALL spHitungBonusLoyalitas('VIP', @poin_awal);
SELECT @poin_awal AS 'Total Poin Setelah Bonus'; -- Output: 30
```

4. KONTROL ALUR LOGIKA (CONTROL FLOW STATEMENTS)

MySQL menyediakan struktur kendali prosedural lengkap di dalam blok BEGIN-END:

- **Pernyataan IF-THEN-ELSE:** Struktur percabangan bersyarat: IF kondisi THEN ... ELSEIF kondisi THEN ... ELSE ... END IF;
- **Pernyataan CASE:** Struktur percabangan bernilai jamak: CASE ekspresi WHEN nilai THEN ... ELSE ... END CASE;
- **Perulangan LOOP-LEAVE:** Perulangan tak terbatas yang dihentikan secara eksplisit menggunakan perintah LEAVE label_loop: [label:] LOOP ... IF kondisi THEN LEAVE [label]; END IF; END LOOP [label];
- **Perulangan WHILE-DO:** Perulangan dengan evaluasi kondisi di awal: WHILE kondisi DO ... END WHILE;
- **Perulangan REPEAT-UNTIL:** Perulangan dengan evaluasi kondisi di akhir: REPEAT ... UNTIL kondisi END REPEAT;

5. CURSOR & ITERASI BARIS DATA DI MYSQL

Secara default, kueri SQL bekerja berbasis himpunan (set-based). Namun, ketika sistem membutuhkan evaluasi logika individual yang berbeda untuk setiap baris data, digunakan CURSOR. Cursor adalah penunjuk memori (pointer) yang menampung baris hasil kueri dan memungkinkan pembacaan data secara sekuensial baris per baris (*row-by-row navigation*).

5.1 Empat Siklus Hidup Cursor

- **1. DECLARE CURSOR:** Mendefinisikan kueri SQL yang akan ditampung ke dalam kursor: DECLARE nama_cursor CURSOR FOR select_statement;
- **2. DECLARE HANDLER:** Mendaftarkan penangan kondisi ketika cursor telah mencapai baris terakhir: DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_selesai = 1;
- **3. OPEN:** Menginisialisasi memori dan mengeksekusi kueri pembentuk cursor: OPEN nama_cursor;
- **4. FETCH & CLOSE:** Mengambil nilai kolom pada baris aktif ke dalam variabel lokal: FETCH nama_cursor INTO var1, var2; dilanjutkan dengan penutupan memori: CLOSE nama_cursor;

5.2 Contoh Nyata: Pembagian Bonus Poin Massal Customer (Modul 4 Halaman 17)

```
DELIMITER $$

CREATE PROCEDURE spProsesBonusPoinMassal()
BEGIN
  DECLARE v_kodecus CHAR(5);
  DECLARE v_poin_saat_ini INT;
  DECLARE v_selesai INT DEFAULT 0;

  -- 1. Deklarasi Cursor untuk memilih customer (Kecuali customer umum M0000)
  DECLARE curCustomer CURSOR FOR
    SELECT kodecus, pointterkumpul
    FROM customer
    WHERE kodecus <> 'M0000';

  -- 2. Deklarasi Handler NOT FOUND
  DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_selesai = 1;

  -- 3. Membuka Cursor
  OPEN curCustomer;

  loopCustomer: LOOP
    -- 4. Mengambil baris data
    FETCH curCustomer INTO v_kodecus, v_poin_saat_ini;

    IF v_selesai = 1 THEN
      LEAVE loopCustomer;
```

```
        END IF;

        -- Evaluasi Aturan Bisnis Bonus Poin
        IF v_poin_saat_ini >= 100 THEN
            UPDATE customer SET pointterkumpul = pointterkumpul + 20 WHERE kodecus = v_kodecus;
        ELSEIF v_poin_saat_ini > 50 THEN
            UPDATE customer SET pointterkumpul = pointterkumpul + 15 WHERE kodecus = v_kodecus;
        ELSEIF v_poin_saat_ini > 20 THEN
            UPDATE customer SET pointterkumpul = pointterkumpul + 10 WHERE kodecus = v_kodecus;
        ELSE
            UPDATE customer SET pointterkumpul = pointterkumpul + 7 WHERE kodecus = v_kodecus;
        END IF;
    END LOOP loopCustomer;

    -- 5. Menutup Cursor
    CLOSE curCustomer;

    SELECT 'Pemrosesan bonus poin seluruh member selesai dieksekusi.' AS status_eksekusi;
END$$

DELIMITER ;

-- Eksekusi:
CALL spProsesBonusPoinMassal();
```

6. PENANGANAN GALAT & INTEGRITAS TRANSAKSI (ERROR HANDLING)

Dalam aplikasi kasir tingkat produksi, kegagalan database engine tidak boleh menyebabkan data tidak konsisten (misal: uang kasir tercatat tetapi stok barang tidak berkurang). MySQL menyediakan sistem penanganan galat terstruktur:

6.1 HANDLER & SIGNAL SQLSTATE

- **DECLARE ... HANDLER:** Mendefinisikan aksi otomatis saat terjadi galat (SQLEXCEPTION) atau peringatan (SQLWARNING). Aksi dapat berupa CONTINUE (melanjutkan baris berikutnya) atau EXIT (segera menghentikan blok BEGIN-END saat itu).
- **SIGNAL SQLSTATE '45000':** Melempar pesan kesalahan kustom ke aplikasi klien. Kode SQLSTATE '45000' adalah kode standar yang dipesan untuk exception pengguna (User-Defined Unhandled Exception).

6.2 Contoh Implementasi: Validasi Stok & Pembatalan Transaksi Kasir

```
DELIMITER $$

CREATE PROCEDURE spKurangiStokBarang(
    IN p_kodeproduk CHAR(4),
    IN p_qty INT
)
BEGIN
    DECLARE v_stok_tersedia INT;

    -- Mengambil stok barang saat ini
    SELECT stok INTO v_stok_tersedia
    FROM produk
    WHERE kodeproduk = p_kodeproduk;
```

```
    -- Validasi 1: Periksa apakah kode produk valid
    IF v_stok_tersedia IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Kesalahan: Kode produk barang tidak ditemukan dalam database.';
    END IF;

    -- Validasi 2: Periksa kecukupan stok
    IF v_stok_tersedia < p_qty THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Transaksi Dibatalkan: Stok produk di gudang tidak mencukupi.';
    ELSE
        -- Update stok fisik produk
        UPDATE produk
        SET stok = stok - p_qty
        WHERE kodeproduk = p_kodeproduk;
    END IF;
END$$

DELIMITER ;

-- Pengujian 1: Stok Cukup (Berhasil)
CALL spKurangiStokBarang('P001', 2);

-- Pengujian 2: Stok Melebihi Batas (Sistem melempar Error 45000)
CALL spKurangiStokBarang('P001', 9999);
-- Respon: ERROR 1644 (45000): Transaksi Dibatalkan: Stok produk di gudang tidak mencukupi.
```

7. STUDI KASUS: PROSEDUR TRANSAKSI CHECKOUT KASIR TERPADU

Untuk mengintegrasikan seluruh materi (UDF, Stored Procedure, Parameter, dan Error Handling), berikut adalah prosedur transaksi checkout penjualan lengkap yang menghubungkan tabel KASIR, CUSTOMER, NOTA, ITEMNOTA, dan PRODUK:

```
DELIMITER $$

CREATE PROCEDURE spCheckoutTransaksi(
    IN p_kodekasir CHAR(3),
    IN p_kodecus CHAR(5),
    IN p_kodeproduk CHAR(4),
    IN p_banyak INT,
    OUT p_nonota_dihasilkan CHAR(10),
    OUT p_total_bayar INT,
    OUT p_poin_baru INT
)
BEGIN
    DECLARE v_harga_satuan INT;
    DECLARE v_stok_tersedia INT;
    DECLARE v_tgl_hari_ini DATE;

    -- Tangani potensi kegagalan integritas dengan ROLLBACK
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
```

```
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    SET v_tgl_hari_ini = CURRENT_DATE();

    -- 1. Validasi Keberadaan Data Produk & Stok
    SELECT harga, stok INTO v_harga_satuan, v_stok_tersedia
    FROM produk WHERE kodeproduk = p_kodeproduk;

    IF v_harga_satuan IS NULL THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: Produk tidak terdaftar.';
    END IF;

    IF v_stok_tersedia < p_banyak THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: Kuantitas beli melebihi persediaan stok toko.';
    END IF;

    -- 2. Generate Nomor Nota Baru secara otomatis
    SET p_nonota_dihasilkan = fcGenerateNoNota(v_tgl_hari_ini);
    SET p_total_bayar = v_harga_satuan * p_banyak;
    SET p_poin_baru = fcHitungPoinBelanja(p_total_bayar);

    -- 3. Memulai Transaksi Atomik
    START TRANSACTION;
        -- Simpan Header Nota
        INSERT INTO nota (nonota, kodekasir, tgltran, kodecus)
        VALUES (p_nonota_dihasilkan, p_kodekasir, v_tgl_hari_ini, p_kodecus);

        -- Simpan Detil Item Nota
        INSERT INTO itemnota (nonota, kodeproduk, banyak, hargajual)
        VALUES (p_nonota_dihasilkan, p_kodeproduk, p_banyak, v_harga_satuan);

        -- Kurangi Stok Barang
        UPDATE produk SET stok = stok - p_banyak WHERE kodeproduk = p_kodeproduk;

        -- Tambahkan Poin Belanja ke Member (Jika bukan customer umum M0000)
        IF p_kodecus <> 'M0000' AND p_poin_baru > 0 THEN
            UPDATE customer SET pointterkumpul = pointterkumpul + p_poin_baru WHERE kodecus = p_kodecus;
        END IF;
    COMMIT;
END$$

DELIMITER ;
```

8. LEMBAR TUGAS & LATIHAN SOAL PRAKTIKUM (UNTUK MAHASISWA)

Petunjuk Pengerjaan Mahasiswa / Praktikan:
1. Pastikan basis data minimarket praktikum (PRODUK, KASIR, CUSTOMER, NOTA, ITEMNOTA) telah aktif.
2. Tuliskan kueri pembuatan Function / Procedure lengkap dengan penyesuaian DELIMITER.
3. Lakukan pengujian eksekusi (CALL / SELECT) dan tangkap layar (screenshot) kode kueri serta output terminal.
4. Perhatikan penggunaan tipe parameter (IN, OUT, INOUT) dan penanganan kondisi error sesuai instruksi soal.

Tingkat 1: User Defined Function (UDF) & Logika Kalkulasi

Soal 1 (UDF Evaluasi Persediaan Stok): Buatlah UDF bernama 'fcCekStatusStok' yang menerima input parameter 'p_kodeproduk' (CHAR(4)) dan mengembalikan nilai skalar string (VARCHAR(20)). Ketentuan: jika stok = 0 kembalikan 'KOSONG', jika stok 1 s.d. 5 kembalikan 'KRITIS', selain itu kembalikan 'TERSEDIA (X Unit)'. Uji function ini menggunakan kueri SELECT pada tabel produk.

Soal 2 (UDF Klasifikasi Member Loyal): Toko ingin mengklasifikasikan jenjang keanggotaan pelanggan. Buatlah UDF bernama 'fcLevelLoyalitas' yang menerima input 'p_point' (INT) dan mengembalikan status (VARCHAR(15)) dengan kriteria: Point >= 100 'PLATINUM', Point 50-99 'GOLD', Point 20-49 'SILVER', dan Point < 20 'BRONZE'. Tampilkan daftar nama customer beserta level loyalitasnya menggunakan fungsi ini.

Soal 3 (UDF Pemformatan Teks Rupiah): Buatlah UDF bernama 'fcFormatRupiah' yang menerima input nominal angka (INT) dan mengembalikan string berformat mata uang Indonesia dengan awalan 'Rp. ' dan pemisah ribuan titik (contoh: 15000 menjadi 'Rp. 15.000'). Uji dengan menampilkan daftar harga produk.

Tingkat 2: Stored Procedure dengan Parameter IN, OUT, dan INOUT

Soal 4 (Procedure Parameter OUT Informasi Customer): Buatlah Stored Procedure bernama 'spCariDataCustomer' dengan satu parameter IN (p_kodecus CHAR(5)) dan tiga parameter OUT (p_nama VARCHAR(50), p_alamat VARCHAR(100), p_poin INT). Procedure bertugas mencari data customer bersangkutan dan menyimpan nilainya ke ketiga parameter OUT. Tulis kueri pemanggilan dan periksa variabel outputnya.

Soal 5 (Procedure Parameter INOUT Diskon Harga): Manajemen minimarket ingin melakukan diskon persentase harga barang. Buatlah Stored Procedure bernama 'spTerapkanDiskonProduk' yang memiliki parameter IN 'p_kodeproduk' (CHAR(4)) dan parameter INOUT 'p_harga' (INT). Nilai harga lama yang dilewatkan ke parameter akan dipotong sebesar 10%, kemudian nilai harga baru tersebut diperbarui ke tabel PRODUK dan nilainya dikembalikan ke pemanggil.

Soal 6 (Procedure Registrasi Staf Kasir): Buatlah Stored Procedure bernama 'spTambahKasirBaru' yang menerima parameter IN nama kasir dan password kasir. Di dalam procedure, generate kode kasir 3 digit otomatis (contoh: '003', '004') dan simpan ke tabel KASIR. Tampilkan data kasir baru yang berhasil ditambahkan.

Tingkat 3: Cursor & Batch Processing Transaksi

Soal 7 (Cursor Kompilasi Riwayat Nota Belanja): Buatlah Stored Procedure bernama 'spKompilasiBelanjaMember' yang memanfaatkan CURSOR untuk mengiterasi seluruh riwayat transaksi belanja seorang customer tertentu (input parameter kodecus). Gabungkan nomor-nomor nota belanja yang pernah dilakukan customer tersebut ke dalam satu variabel string panjang yang dipisahkan oleh tanda koma, lalu tampilkan hasilnya.

Soal 8 (Cursor Pembaruan Stok Masal): Toko ingin merestok seluruh barang yang memiliki stok di bawah 10 unit. Buatlah Stored Procedure 'spRestokOtomatis' yang menggunakan CURSOR untuk mencari semua produk berkategori 'stok kritis' (< 10) dan secara otomatis menambahkan 50 unit ke masing-masing produk tersebut. Catat dan tampilkan berapa banyak produk yang berhasil direstok.

Tingkat 4: Error Handling & Complex Transaction Control

Soal 9 (Validasi Integritas Hapus Data dengan SIGNAL): Buatlah Stored Procedure 'spHapusCustomerAman' yang menerima kode customer. Procedure harus memvalidasi: jika customer tersebut pernah memiliki riwayat transaksi pada tabel NOTA, batalkan operasi dan lemparkan error kustom via SIGNAL SQLSTATE '45000' dengan pesan: 'Tidak dapat menghapus customer: Data customer terikat dengan riwayat nota transaksi'. Jika tidak memiliki nota, lakukan penghapusan.

Soal 10 (Prosedur Transaksi Atomik Penjualan Barang): Rancanglah Stored Procedure 'spPencatatanItemTransaksi' yang menerima parameter: nomor nota, kode produk, dan jumlah beli. Gunakan HANDLER FOR SQLEXCEPTION untuk menjamin atomisitas data. Di dalam procedure: periksa kecukupan stok di tabel PRODUK, kurangi stok barang, dan simpan baris transaksi ke tabel ITEMNOTA. Jika stok tidak cukup, batalkan transaksi dan lempar error kustom.

9. LAMPIRAN: KUNCI JAWABAN & PEMBAHASAN LENGKAP (KHUSUS DOSEN & ASISTEN PRAKTIKUM)

Bagian ini memuat seluruh solusi kueri SQL baku, sintaks pengujian eksekusi, serta analisis logika teknis untuk instrumen evaluasi penilaian asisten praktikum:

Solusi & Pembahasan Soal 1 (fcCekStatusStok)

```
DELIMITER $$
CREATE FUNCTION fcCekStatusStok(p_kodeproduk CHAR(4))
RETURNS VARCHAR(30)
DETERMINISTIC
READS SQL DATA
BEGIN
   DECLARE v_stok INT;
   DECLARE v_status VARCHAR(30);

   SELECT stok INTO v_stok FROM produk WHERE kodeproduk = p_kodeproduk;

   IF v_stok IS NULL THEN
      SET v_status = 'PRODUK TIDAK ADA';
   ELSEIF v_stok = 0 THEN
      SET v_status = 'KOSONG';
   ELSEIF v_stok <= 5 THEN
      SET v_status = 'KRITIS';
   ELSE
      SET v_status = CONCAT('TERSEDIA (', v_stok, ' Unit)');
   END IF;

   RETURN v_status;
END$$
DELIMITER ;

-- Kueri Pengujian:
SELECT kodeproduk, namaproduk, stok, fcCekStatusStok(kodeproduk) AS status_stok FROM produk;
```

Analisis Logika: Analisis: Fungsi membaca data stok dari tabel produk dan mengevaluasi ambang batas menggunakan struktur IF-ELSEIF.

Solusi & Pembahasan Soal 2 (fcLevelLoyalitas)

```
DELIMITER $$
CREATE FUNCTION fcLevelLoyalitas(p_point INT)
RETURNS VARCHAR(15)
DETERMINISTIC
NO SQL
BEGIN
   DECLARE v_level VARCHAR(15);
   IF p_point >= 100 THEN
      SET v_level = 'PLATINUM';
   ELSEIF p_point >= 50 THEN
      SET v_level = 'GOLD';
   ELSEIF p_point >= 20 THEN
      SET v_level = 'SILVER';
```

```
    ELSE
        SET v_level = 'BRONZE';
    END IF;
    RETURN v_level;
END$$
DELIMITER ;

-- Kueri Pengujian:
SELECT kodecus, namacus, pointterkumpul, fcLevelLoyalitas(pointterkumpul) AS tier_loyalitas FROM customer;
```

Analisis Logika: Analisis: Menggunakan karakteristik NO SQL karena murni komputasi skalar tanpa mengakses tabel.

Solusi & Pembahasan Soal 3 (fcFormatRupiah)

```
DELIMITER $$
CREATE FUNCTION fcFormatRupiah(p_nominal INT)
RETURNS VARCHAR(30)
DETERMINISTIC
NO SQL
BEGIN
    RETURN CONCAT('Rp. ', REPLACE(FORMAT(p_nominal, 0), ',', '.'));
END$$
DELIMITER ;

-- Kueri Pengujian:
SELECT kodeproduk, namaproduk, fcFormatRupiah(harga) AS harga_rupiah FROM produk;
```

Analisis Logika: Analisis: Memanfaatkan fungsi bawaan FORMAT() dan REPLACE() untuk mengubah koma standar internasional menjadi titik ribuan khas Indonesia.

Solusi & Pembahasan Soal 4 (spCariDataCustomer)

```
DELIMITER $$
CREATE PROCEDURE spCariDataCustomer(
    IN p_kodecus CHAR(5),
    OUT p_nama VARCHAR(50),
    OUT p_alamat VARCHAR(100),
    OUT p_poin INT
)
BEGIN
    SELECT namacus, almtcus, pointterkumpul
    INTO p_nama, p_alamat, p_poin
    FROM customer
    WHERE kodecus = p_kodecus;
END$$
DELIMITER ;

-- Kueri Pengujian:
CALL spCariDataCustomer('M0001', @nama, @alamat, @poin);
SELECT @nama AS Nama, @alamat AS Alamat, @poin AS Poin;
```

Analisis Logika: Analisis: Klausa SELECT INTO memetakan kolom baris kueri secara langsung ke dalam parameter OUT.

Solusi & Pembahasan Soal 5 (spTerapkanDiskonProduk)

```
DELIMITER $$
CREATE PROCEDURE spTerapkanDiskonProduk(
   IN p_kodeproduk CHAR(4),
   INOUT p_harga INT
)
BEGIN
   -- Potong harga sebesar 10%
   SET p_harga = ROUND(p_harga * 0.90);

   -- Update ke tabel fisik produk
   UPDATE produk SET harga = p_harga WHERE kodeproduk = p_kodeproduk;
END$$
DELIMITER ;

-- Kueri Pengujian:
SET @harga_awal = 10000;
CALL spTerapkanDiskonProduk('P001', @harga_awal);
SELECT @harga_awal AS harga_setelah_diskon;
```

Analisis Logika: Analisis: Parameter INOUT membawa nilai harga awal dari pemanggil, dimutasi 10% di dalam procedure, dan nilainya dikembalikan.

Solusi & Pembahasan Soal 6 (spTambahKasirBaru)

```
DELIMITER $$
CREATE PROCEDURE spTambahKasirBaru(
   IN p_nama VARCHAR(50),
   IN p_pass CHAR(6)
)
BEGIN
   DECLARE last_code INT DEFAULT 0;
   DECLARE new_kodekasir CHAR(3);

   -- Validasi panjang password
   IF CHAR_LENGTH(p_pass) <> 6 THEN
      SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: Password kasir harus tepat 6 karakter.';
   END IF;

   -- Generate Kode Kasir 3 digit otomatis
   SELECT IFNULL(MAX(CAST(kodekasir AS UNSIGNED)), 0) INTO last_code FROM kasir;
   SET new_kodekasir = LPAD(last_code + 1, 3, '0');

   INSERT INTO kasir (kodekasir, namakasir, pass) VALUES (new_kodekasir, p_nama, p_pass);
   SELECT * FROM kasir WHERE kodekasir = new_kodekasir;
END$$
DELIMITER ;

-- Kueri Pengujian:
CALL spTambahKasirBaru('Doni Pratama', '123456');
```

Analisis Logika: Analisis: Menjamin kepatuhan integritas panjang password kasir (6 karakter) dan auto-numbering kode kasir.

Solusi & Pembahasan Soal 7 (spKompilasiBelanjaMember)

```
DELIMITER $$
CREATE PROCEDURE spKompilasiBelanjaMember(IN p_kodecus CHAR(5))
BEGIN
   DECLARE v_nota CHAR(10);
   DECLARE v_selesai INT DEFAULT 0;
   DECLARE v_daftar VARCHAR(1000) DEFAULT '';

   DECLARE curNota CURSOR FOR
      SELECT nonota FROM nota WHERE kodecus = p_kodecus;
   DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_selesai = 1;

   OPEN curNota;
   readLoop: LOOP
      FETCH curNota INTO v_nota;
      IF v_selesai = 1 THEN
         LEAVE readLoop;
      END IF;
      IF v_daftar = '' THEN
         SET v_daftar = v_nota;
      ELSE
         SET v_daftar = CONCAT(v_daftar, ', ', v_nota);
      END IF;
   END LOOP readLoop;
   CLOSE curNota;

   SELECT p_kodecus AS Member, IFNULL(NULLIF(v_daftar, ''), 'Belum Pernah Belanja') AS Riwayat_Nota;
END$$
DELIMITER ;

-- Kueri Pengujian:
CALL spKompilasiBelanjaMember('M0001');
```

Analisis Logika: Analisis: Menggunakan cursor untuk menggabungkan sekumpulan nomor nota belanja menjadi satu baris string.

Solusi & Pembahasan Soal 8 (spRestokOtomatis)

```
DELIMITER $$
CREATE PROCEDURE spRestokOtomatis(OUT p_jml_produk_direstok INT)
BEGIN
   DECLARE v_kd CHAR(4);
   DECLARE v_selesai INT DEFAULT 0;
   DECLARE v_counter INT DEFAULT 0;

   DECLARE curKritis CURSOR FOR
      SELECT kodeproduk FROM produk WHERE stok < 10;
   DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_selesai = 1;
```

```
    OPEN curKritis;
    restokLoop: LOOP
        FETCH curKritis INTO v_kd;
        IF v_selesai = 1 THEN
            LEAVE restokLoop;
        END IF;

        UPDATE produk SET stok = stok + 50 WHERE kodeproduk = v_kd;
        SET v_counter = v_counter + 1;
    END LOOP restokLoop;
    CLOSE curKritis;

    SET p_jml_produk_direstok = v_counter;
END$$
DELIMITER ;

-- Kueri Pengujian:
CALL spRestokOtomatis(@total_direstok);
SELECT @total_direstok AS 'Jumlah Produk Berhasil Direstok';
```

Analisis Logika: Analisis: Cursor mengidentifikasi seluruh produk dengan stok kritis dan melakukan penambahan batch 50 unit.

Solusi & Pembahasan Soal 9 (spHapusCustomerAman)

```
DELIMITER $$
CREATE PROCEDURE spHapusCustomerAman(IN p_kodecus CHAR(5))
BEGIN
    DECLARE v_jml_nota INT DEFAULT 0;

    -- Hitung transaksi terkait
    SELECT COUNT(*) INTO v_jml_nota FROM nota WHERE kodecus = p_kodecus;

    IF v_jml_nota > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Tidak dapat menghapus customer: Data customer terikat dengan riwayat nota transaksi.';
    ELSE
        DELETE FROM customer WHERE kodecus = p_kodecus;
        SELECT CONCAT('Customer ', p_kodecus, ' berhasil dihapus.') AS status;
    END IF;
END$$
DELIMITER ;

-- Kueri Pengujian (Customer yang memiliki nota M0001 -> Ditolak):
CALL spHapusCustomerAman('M0001');
-- Hasil: ERROR 1644 (45000): Tidak dapat menghapus customer: Data customer terikat dengan riwayat nota transaksi.
```

Analisis Logika: Analisis: Mencegah error foreign key cascaded yang tidak terkontrol dengan validasi logika ramah pengguna via SIGNAL.

Solusi & Pembahasan Soal 10 (spPencatatanItemTransaksi)

```
DELIMITER $$
CREATE PROCEDURE spPencatatanItemTransaksi(
   IN p_nonota CHAR(10),
   IN p_kodeproduk CHAR(4),
   IN p_qty INT
)
BEGIN
   DECLARE v_stok INT DEFAULT 0;
   DECLARE v_harga INT DEFAULT 0;

   -- Rollback otomatis jika ada error database
   DECLARE EXIT HANDLER FOR SQLEXCEPTION
   BEGIN
      ROLLBACK;
      RESIGNAL;
   END;

   SELECT stok, harga INTO v_stok, v_harga FROM produk WHERE kodeproduk = p_kodeproduk;

   IF v_stok IS NULL THEN
      SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: Produk tidak ditemukan.';
   END IF;

   IF v_stok < p_qty THEN
      SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: Kuantitas melebihi ketersediaan stok fisik toko.';
   END IF;

   START TRANSACTION;
      INSERT INTO itemnota (nonota, kodeproduk, banyak, hargajual)
      VALUES (p_nonota, p_kodeproduk, p_qty, v_harga);

      UPDATE produk SET stok = stok - p_qty WHERE kodeproduk = p_kodeproduk;
   COMMIT;

   SELECT 'Item nota berhasil dicatat dan stok fisik terpotong.' AS status;
END$$
DELIMITER ;
```

Analisis Logika: Analisis: Menggunakan blok transaksi START TRANSACTION - COMMIT dengan jaminan atomisitas via DECLARE EXIT HANDLER FOR SQLEXCEPTION.

10. DAFTAR PUSTAKA & REFERENSI RESMI

- **1.** Oracle Corporation. (2024). MySQL 8.0 Reference Manual: Stored Routines & Cursor Management. https://dev.mysql.com/doc/refman/8.0/en/stored-routines.html
- **2.** Carlos Coronel & Steven Morris. (2019). Database Systems: Design, Implementation, & Management (13th Edition). Cengage Learning, Boston, USA.
- **3.** Paul DuBois. (2014). MySQL Developer's Library (5th Edition). Addison-Wesley Professional.
- **4.** Laboratorium Sistem Informasi. (2024). Modul Praktikum Sistem Manajemen Basis Data Lanjut: Materi Praktikum 4. Universitas Amikom Yogyakarta.